

ESP32 16-Channel Smart Relay Controller

Version: 9.0

Author: Raff Alds

Repository: <https://www.github.com/xiv3r>

License: GPLv3

Target Platform: ESP32 Microcontroller

Table of Contents

1. [Overview](#)
2. [Key Features](#)
3. [Hardware Configuration](#)
4. [System Architecture](#)
5. [Constants & Definitions](#)
6. [Data Structures](#)
7. [Core Functions](#)
8. [WiFi Management Functions](#)
9. [Time Management Functions](#)
10. [RTC Functions](#)
11. [Relay Control Functions](#)
12. [Schedule Functions](#)
13. [Web Server Functions](#)
14. [REST API Endpoints](#)
15. [Self-Healing System](#)
16. [Memory Management](#)

17. [Configuration Management](#)

18. [System Recovery](#)

Overview

This is a **production-grade, unattended relay controller firmware** for ESP32 microcontroller designed for long-term autonomous operation (7+ years target lifespan). The system manages up to **16 independently controlled relays** with:

- **Redundant time sources:** NTP, DS3231 hardware RTC, and browser-based time sync
 - **Persistent configuration:** NVS (Non-Volatile Storage) with write optimization
 - **Self-healing recovery:** Automatic system restoration on WiFi/time failures
 - **Flexible scheduling:** Per-relay time-based and day-of-week/month schedules
 - **RESTful API:** Web interface for remote management
 - **Dual-mode WiFi:** STA (Station) + AP (Access Point) simultaneous operation
 - **Year 2106 support:** Extended Unix timestamp validation
-

Key Features

Network & Connectivity

- **Dual WiFi Mode:** Operates as both WiFi Station and Access Point
- **NTP Time Sync:** Automatic synchronization with multiple NTP servers (Google, Windows, Cloudflare, Facebook)

- **mDNS Support:** Hostname-based discovery (e.g., `esp32.local`)
- **DNS Server:** Captive portal support with custom DNS
- **Auto-Reconnection:** Smart WiFi reconnection with exponential backoff

Time Management

- **NTP Synchronization:** 3600-second interval with 4 fallback servers
- **Hardware RTC (DS3231):** Battery-backed real-time clock for persistent timekeeping
- **Internal RTC:** Software-based epoch tracking with drift compensation
- **Browser Time Sync:** Manual time synchronization from web interface
- **Millis() Rollover Safety:** Elapsed-time patterns prevent 49-day overflow bugs

Relay Control

- **16 Independent Relays:** Fully configurable GPIO pin assignment
- **Manual Override:** Per-relay on/off control with persistent state
- **Automatic Scheduling:** 8 independent timer schedules per relay
- **Active-Low Support:** Configurable relay polarity (active-low or active-high)
- **Day-of-Week Scheduling:** All days, weekdays, weekends, or custom combinations
- **Monthly Day Scheduling:** Bitmask-based selection of specific days (1-31)
- **Global Active Mode:** 3 modes for managing relay behavior across system

Persistence & Recovery

- **NVS Flash Storage:** Configuration stored in encrypted NVS namespace
- **Critical State Backup:** Checksum-protected relay state and timing info
- **Self-Healing Recovery:** Automatic recovery from WiFi/RTC/NTP failures
- **Factory Reset:** Hold boot button 5 seconds for complete reset
- **Write Optimization:** Dirty-flag batching reduces flash wear from ~6 months to 19 years

System Health

- **Memory Management:** Periodic cleanup to prevent heap fragmentation
- **Connection Monitoring:** Last-activity tracking and timeout detection
- **Web Server Health Check:** Automatic restart if unresponsive
- **Health Metrics:** Tracks NTP syncs, WiFi reconnections, memory usage
- **Boot Button Monitor:** Real-time factory reset detection

Hardware Configuration

Default Relay GPIO Pin Mapping

Relay	GPIO	Relay	GPIO
1	15	9	23
2	2	10	13
3	4	11	14

Relay	GPIO	Relay	GPIO
4	5	12	27
5	18	13	26
6	19	14	25
7	3	15	33
8	1	16	32

Note: GPIO pins are fully reconfigurable via REST API

Connected Hardware

- **Primary:** ESP32 Microcontroller (dual-core, 240 MHz)
- **RTC Backup:** DS3231 Real-Time Clock (I2C, battery-backed)
- **Storage:** NVS (internal flash) + LittleFS support
- **Relays:** 16x 5V/12V relay modules (GPIO output controlled)

I2C Configuration

- **I2C Bus:** Standard ESP32 I2C pins (SDA, SCL)
- **DS3231 Address:** 0x68 (standard)
- **Pullup Resistors:** Assumed present on board

System Architecture

Timing Intervals

Function	Interval	Purpose
NTP Retry	30 sec	Retry failed NTP syncs

Function	Interval	Purpose
WiFi Check	10 sec	Monitor WiFi connectivity
WiFi Timeout	20 sec	Maximum connection wait
RTC Update	100 ms	Update internal clock
Schedule Process	250 ms	Check active schedules
Relay Update	500 ms	Apply relay changes
RTC Rebase	5 min	Sync internal RTC drift
RTC/DS3231 Sync	1 hour	Full RTC synchronization
Memory Cleanup	30 sec	Garbage collection
Memory Check	60 sec	Heap fragmentation test

WiFi Scanning

- **Pause Duration:** 15 seconds (WiFi suspended during scan)
- **Auto-Resume:** Automatic reconnection after scan
- **Connection Timeout:** 10 seconds per connection attempt

Boot Sequence

- **Factory Reset Hold:** 5 seconds on boot button (GPIO 0)
- **Factory Reset Behavior:** Clears all config, resets to defaults

Constants & Definitions

Unix Time Validation

```
#define MAX_UNIX_TIME 4294967295UL // Year 2106
support
#define MIN_UNIX_TIME 1000000000UL // Year 2001
minimum
#define VALID_UNIX_TIME(epoch) ((epoch) >
MIN_UNIX_TIME && (epoch) < MAX_UNIX_TIME)
```

Day-of-Week Bitmask

```
#define DAY_SUNDAY (1 << 0) // 0x01
#define DAY_MONDAY (1 << 1) // 0x02
#define DAY_TUESDAY (1 << 2) // 0x04
#define DAY_WEDNESDAY (1 << 3) // 0x08
#define DAY_THURSDAY (1 << 4) // 0x10
#define DAY_FRIDAY (1 << 5) // 0x20
#define DAY_SATURDAY (1 << 6) // 0x40
#define DAY_ALL 0x7F // All days
#define DAY_WEEKDAYS 0x3E // Mon-Fri
#define DAY_WEEKENDS 0x41 // Sat-Sun
```

NTP Server Pool

```
1. time.google.com
2. time.windows.com
3. time.cloudflare.com
4. time.facebook.com
```

NVS Storage

- **Namespace:** "relay16"
- **Magic Number:** 0x1234 (config validation)
- **Version:** 9 (current firmware version)
- **GPIO Config Magic:** 0xD002
- **Extended Config Magic:** 0xEC

Time Source Enumeration

```
TIME_SOURCE_NONE    = 0  (no valid time)
TIME_SOURCE_NTP     = 1  (NTP synchronized)
TIME_SOURCE_BROWSER = 2  (browser sync)
TIME_SOURCE_RTC     = 3  (RTC backed-up)
```

NTP State Machine

```
NTP_STATE_IDLE      = 0  (ready for next sync)
NTP_STATE_CONNECTING = 1  (resolving server)
NTP_STATE_WAITING   = 2  (awaiting response)
NTP_STATE_TIMEOUT   = 5  sec timeout
```

Data Structures

TimerSchedule

Defines 8 independent timer schedules per relay with day/time control.

```
struct TimerSchedule {
    uint8_t startHour[8], startMinute[8],
    startSecond[8]; // Start times
    uint8_t stopHour[8], stopMinute[8],
    stopSecond[8]; // Stop times
    bool    enabled[8];
    // Per-schedule enable
    uint8_t days[8];
    // Day-of-week mask
    uint32_t monthDays[8];
    // Monthly day bitmask (1-31)
};
```


Usage: Each relay can have up to 8 independent schedules active simultaneously (OR logic).

RelayConfig

Per-relay configuration and state.

```
struct RelayConfig {
    char    name[16];           // Relay
display name
    TimerSchedule schedule;     // 8 timer
schedules
    bool    manualOverride;     // Manual
control active
    bool    manualState;       // Manual
on/off state
    // Padding to fixed size
};
```

Storage: 16 instances in NVS (one per relay)

SystemConfig

Global system configuration.

```
struct SystemConfig {
    char    ssid[32];          // WiFi
SSID
    char    password[64];      // WiFi
password
    char    ntpServer[64];     // Primary
NTP server
    uint16_t ntpPort;          // NTP
port (default 123)
    int8_t  timezoneOffset;    // UTC
offset in hours (-12 to +14)
```

```

    bool    daylightSaving;           // DST
enabled
    bool    apEnabled;               // AP mode
active
    // Validation fields
};

```

Storage: Single instance in NVS, ~250 bytes

ExtConfig

Extended configuration (v2).

```

struct ExtConfig {
    uint8_t  global_active_mode;      // Global
relay behavior mode (0-2)
    uint8_t  padding[16];             //
Reserved for future use
};

```

Modes:

- 0: Standard (schedules + manual override)
- 1: Always-on (ignore schedules)
- 2: Always-off (ignore schedules)

HealthMetrics

System performance tracking.

```

struct HealthMetrics {
    uint32_t ntpSyncCount;            // Total
successful NTP syncs
    uint32_t wifiReconnectCount;      // Total
WiFi reconnections
    uint32_t memoryFragmentation;     // Current
};

```

```
fragmentation %
    time_t    lastNTPSync;           //
Timestamp of last NTP sync
    time_t    lastWiFiReconnect;     //
Timestamp of last WiFi reconnection
};
```

CriticalRelayState

Checksum-protected relay state for recovery.

```
struct CriticalRelayState {
    uint8_t count;                   // Active
relay count
    bool    states[MAX_RELAYS];     // Current
relay states
    time_t  lastUpdateTime;         // Last
state change timestamp
    uint32_t checksum;              // CRC32
for validation
};
```

ResponseCache

API response caching to reduce processing.

```
struct ResponseCache {
    char    buffer[2048];           // Cached
JSON response
    unsigned long timestamp;        // Cache
creation time
    bool    valid;                  // Cache
validity flag
};
```

GPIOPinConfig

Dynamic GPIO pin configuration.

```
struct GPIOPinConfig {
    uint8_t pins[MAX_RELAYS];           // GPIO pin
numbers
    uint8_t count;                       // Active
relay count
    uint16_t magic;                     //
Validation magic
    bool activeLow[MAX_RELAYS];         // Polarity
per relay (true=LOW=ON)
};
```

SelfHealingSystem

Recovery management class.

```
class SelfHealingSystem {
    uint8_t subsystemHealthFlags;       // Bitmask
of subsystem health
    unsigned long lastHealthCheck;     // Last
health check time

    // Methods documented in Self-Healing System
section
};
```

Core Functions

System Initialization

```
void setup()
```

Location: Line 2577

Frequency: Once at boot

Purpose: Complete system initialization sequence

Operations:

1. Initialize serial communication (115200 baud)
2. Initialize I2C (GPIO 21=SDA, GPIO 22=SCL) for DS3231
3. Load GPIO configuration from NVS
4. Initialize relay GPIO pins (OUTPUT mode)
5. Load system configuration from NVS
6. Load extended configuration
7. Initialize internal RTC state
8. Attempt to load RTC from DS3231 battery backup
9. Setup NTP client with fallback servers
10. Initialize mDNS service
11. Setup DNS server (port 53)
12. Setup web server (port 80) and register all endpoints
13. Start WiFi connection sequence
14. Start mDNS advertising
15. Reset relay outputs to safe state
16. Initialize critical state checksum
17. Start schedule cache update
18. Initialize self-healing system

Pin Initialization:

- Boot button (GPIO 0): INPUT_PULLUP for factory reset
- I2C: GPIO 21 (SDA), GPIO 22 (SCL)
- Relays: Configurable (default pins 1-4, 5-8, 13-14, etc.)

Safety Checks:

- All relays default to OFF state

- Manual override cleared at boot
 - Schedule cache rebuilt
 - AP mode stays enabled regardless of STA status
-

void initDefaults()

Location: Line 2801

Frequency: Once (factory reset or first boot)

Purpose: Initialize all system settings to factory defaults

Default Values Set:

- SSID: "ESP32_16CH_Timer_Switch"
- AP Password: "ESP32-admin"
- Timezone: UTC+0
- NTP Server: Primary from pool
- All relays: Disabled, default names ("Relay 1" through "Relay 16")
- All schedules: Enabled for DAY_ALL (no time restrictions)
- Manual override: Disabled for all relays
- GPIO pins: Default pin assignment table
- Active-low: true for all relays

NVS Keys Created:

- "gpio_config" : GPIO pin configuration
 - "sys_config" : System settings
 - "ext_config" : Extended features
 - "relay_0" through "relay_15" : Per-relay configs
-

Main Loop

`void loop()`

Location: Line 2657

Frequency: Continuous (Arduino loop)

Purpose: Main event loop managing all subsystems

Scheduled Operations (based on elapsed time):

Task	Interval	Handler
Boot button check	N/A	<code>checkBootButton()</code>
WiFi health monitoring	10 sec	Monitor connection state
NTP synchronization	30 sec	<code>tryNTPSync()</code>
Browser time sync check	N/A	<code>handleBrowserTimeSync()</code>
RTC rebase	5 min	<code>performRTCRebase()</code>
RTC/DS3231 full sync	1 hour	<code>immediateDS3231Sync()</code>
Relay schedule processing	250 ms	<code>processRelaySchedules()</code>
Relay output updates	500 ms	<code>updateRelayOutputs()</code>
Memory cleanup	30 sec	<code>performMemoryCleanup()</code>
Memory health check	60 sec	<code>checkAndCleanMemory()</code>

Task	Interval	Handler
Web server health check	N/A	<code>checkWebServerHealth()</code>
Self-healing recovery	Adaptive	<code>smartRecovery()</code>
DNS server processing	Continuous	<code>dnsServer.processNextRequest()</code>
Web server client handling	Continuous	<code>server.handleClient()</code>
RTC internal clock update	100 ms	Increment internal epoch

Rollover-Safe Timing:

All comparisons use `(now - last) >= interval` pattern to handle `millis()` overflow at 49 days.

WiFi Management Functions

WiFi Connection & Status

```
void setWiFiStationEnabled(bool enabled)
```

Location: Line 485

Purpose: Enable/disable WiFi Station mode (client connection)

Parameters:

- `enabled: true` to connect to configured SSID, `false` to disconnect

Operations:

- If `true` :
 1. Set WiFi mode to `WIFI_AP_STA` (concurrent AP + STA)
 2. Store SSID and password into WiFi stack
 3. Begin connection sequence
 4. Flag `wifiConnecting = true`
 5. Reset reconnection counter
 6. Record connection attempt timestamp
- If `false` :
 1. Disconnect from current network
 2. Set WiFi mode to `WIFI_AP` (AP only, STA disabled)
 3. Flag `wifiConnected = false`
 4. Clear reconnection counter

Safety:

- AP mode always remains active (`WIFI_AP_STA`)
 - Never disables AP interface (captive portal stays available)
 - Connection timeout: 20 seconds
-

`void beginWiFiConnect()`

Location: Line 2338

Frequency: Called every 10 seconds if not connected

Purpose: Attempt WiFi station connection with exponential backoff

Logic:

1. Check elapsed time since last connection attempt
2. If connected: reset attempt counter, skip
3. If not connecting and enough time passed:
 - Initiate WiFi connection (`wifi.begin()`)

- Start 20-second timeout counter
- Increment attempt counter
- Delay based on attempt count (exponential backoff)

Reconnection Backoff:

- Attempt 1-3: Immediate retry
- Attempt 4-6: Exponential delay increases
- Attempt 7+: Maximum backoff (then reset)

WiFi Scanning

`void pauseWiFiForScan()`

Location: Line 685

Purpose: Pause WiFi operations during network scan

Operations:

1. Disconnect WiFi STA temporarily
2. Set `wifiPausedForScan = true`
3. Record pause start time
4. Set pause duration to 15 seconds

Automatic Resume:

- Checked in main loop every iteration
- After 15 seconds, WiFi automatically reconnects
- Works only if not already scanning

`void handleWiFiScanStart()`

Location: Line 3254

HTTP: `POST /api/wifi/scan/start`

Purpose: Initiate asynchronous WiFi network scan

Operations:

1. Update last connection activity timestamp
2. Pause WiFi to avoid conflicts
3. Start async network scan
4. Return JSON: `{"scanning":true}`

Note: Scan completes asynchronously; use `handleWiFiScanResults()` to poll results.

`void handleWiFiScanResults()`

Location: Line 3274

HTTP: `GET /api/wifi/scan/results`

Purpose: Retrieve cached WiFi scan results

Response:

```
{
  "scanning": false,
  "networks": [
    {"ssid": "Network1", "rssi": -45, "secured": true},
    {"ssid": "Network2", "rssi": -67, "secured": false}
  ]
}
```

Fields:

- `scanning : true` if scan still in progress
- `networks[].ssid` : Network name
- `networks[].rssi` : Signal strength (-120 to 0 dBm)
- `networks[].secured` : WPA/WEK protection active

WiFi Configuration

`void handleGetWiFi()`

Location: Line 3187

HTTP: GET /api/wifi

Purpose: Retrieve current WiFi station configuration and status

Response:

```
{
  "ssid": "MyNetwork",
  "connected": true,
  "ip": "192.168.1.100",
  "rssi": -55,
  "staEnabled": true,
  "apIp": "192.168.4.1",
  "apGateway": "192.168.4.1",
  "apNetmask": "255.255.255.0"
}
```

Fields:

- `ssid` : Currently configured network name
- `connected` : WiFi connection status
- `ip` : Current station IP address
- `rssi` : Signal strength (dBm)
- `staEnabled` : Station mode active
- `apIp/apGateway/apNetmask` : AP network info

`void handleSaveWiFi()`

Location: Line 3200

HTTP: POST /api/wifi

Content-Type: application/json

Purpose: Update WiFi station credentials and attempt connection

Request Body:

```
{
  "ssid": "NewNetwork",
  "password": "SecurePass123",
  "stationEnabled": true
}
```

Operations:

1. Validate JSON structure
2. Save SSID/password to system config NVS
3. If stationEnabled=true, initiate connection
4. If stationEnabled=false, disable station mode
5. Start 20-second connection timeout
6. Return connection status

Response:

```
{
  "success": true,
  "ssid": "NewNetwork",
  "connecting": true
}
```

Validation:

- SSID length: 1-32 characters
- Password length: 8-64 characters
- Invalid creds return 400 error

WiFi Status

```
void handleGetWiFi()
```

HTTP: GET /api/wifi

Purpose: Retrieve WiFi status without modification

Response includes:

- Current SSID
 - Connection status
 - IP address
 - Signal strength (RSSI)
 - AP IP and subnet info
-

Time Management Functions

NTP Synchronization

```
void tryNTPSync()
```

Location: Line 2224

Frequency: Every 30 seconds (or on timeout)

Purpose: Attempt asynchronous NTP time synchronization

State Machine:

```
graph LR
  IDLE --> CONNECTING
  CONNECTING --> WAITING
  WAITING -- success --> IDLE
  WAITING -- timeout/retry --> IDLE
```

Operations:

1. IDLE State:

- If 30+ seconds since last attempt
- Pick next NTP server from pool (round-robin)

- Update client to server address
- Begin async NTP request
- Change to CONNECTING state

2. **CONNECTING State:**

- Wait for DNS resolution
- After success, change to WAITING
- 5-second timeout for DNS

3. **WAITING State:**

- Poll for NTP response
- If response received:
 - Extract UTC epoch
 - Validate epoch range (MIN_UNIX_TIME to MAX_UNIX_TIME)
 - Call `syncInternalRTC(epoch)`
 - Update `timeSource = TIME_SOURCE_NTP`
 - Record sync timestamp
 - Reset server failure counter
 - Change to IDLE state
- If 5+ seconds without response:
 - Increment NTP server failure counter
 - If counter > 3: advance to next server
 - Retry with new server
 - Reset timeout

Server Failover:

- Up to 3 failures per server before switching
- Cycle through 4 servers (Google, Windows, Cloudflare, Facebook)
- After 4th server timeout: wait 30 seconds and restart

Health Tracking:

- Increments `healthMetrics.ntpSyncCount` on success
 - Records `healthMetrics.lastNTPSync` timestamp
 - Logs failures for debugging
-

```
void syncInternalRTC(time_t rawUtcEpoch)
```

Location: Line 2196

Purpose: Update internal software RTC with new epoch time

Parameters:

- `rawUtcEpoch`: Unix timestamp in UTC (seconds since 1970-01-01)

Operations:

1. Validate epoch is in range (1 billion to 4.2 billion)
2. Calculate timezone-adjusted local epoch: `internalEpoch = rawUtcEpoch + (timezoneOffset * 3600) + (daylightSaving ? 3600 : 0)`
3. Record sync timestamp: `lastRTCSync = millis()`
4. Initialize drift compensation: `driftCompensation = 1.0f`
5. Reset drift history: `ntpDriftHistory[0..3] = 1.0f`
6. Synchronize to DS3231 (if present and ready)
7. Reset RTC rebase counter
8. Update `rtcTimeValid = true`

Drift Compensation:

- Tracks milliseconds vs expected RTC drift
 - Maintains 4-sample history of correction factors
 - Averages history to smooth estimates
-

`void handleSyncNTP()`

Location: Line 3353

HTTP: POST /api/time/sync/ntp

Purpose: Manually trigger immediate NTP synchronization

Operations:

1. Call `tryNTPSync()` immediately
2. Force NTP state to IDLE for immediate retry
3. Record manual sync attempt
4. Return JSON with current time source

Response:

```
{
  "success": true,
  "syncing": true,
  "timeSource": "ntp"
}
```

`void handleBrowserTimeSync()`

Location: Line 2276

Frequency: Continuous (checked every loop iteration)

Purpose: Process time updates from web browser

Operations:

1. Check for pending browser time sync request
2. If received from web API (/api/time/browser-sync):
 - Extract epoch from request
 - Validate against current time (reject if > 1 hour difference)
 - If valid:
 - Call `syncInternalRTC(browserEpoch)`

- Set `timeSource = TIME_SOURCE_BROWSER`
- Record timestamp: `lastBrowserSync = millis()`
- Update schedule cache
- If invalid:
 - Reject with error response

Security:

- Only accepts time within ± 1 hour of current estimate
 - Prevents obvious invalid times
 - Logs browser sync events
-

Time Retrieval

`time_t getCurrentEpoch()`

Location: Line 585

Return Type: `time_t` (Unix timestamp)

Purpose: Get current time with automatic drift compensation

Logic:

1. Get current millis: `now = millis()`
2. Calculate elapsed: `elapsed = (now - lastRTCSync)` (rollover-safe)
3. Apply drift compensation: `elapsed *= driftCompensation`
4. Current epoch = `internalEpoch + (elapsed / 1000.0)`
5. Return as integer (rounded to whole second)

Accuracy:

- ± 100 ms typical (depends on drift correction)
- Self-corrects every RTC rebase (5 minutes)
- Full resync every 1 hour

```
inline time_t getLocalEpoch(time_t utcEpoch)
```

Location: Line 435

Purpose: Convert UTC epoch to local time with timezone + DST

Formula:

```
localEpoch = utcEpoch + (timezoneOffset * 3600) +  
(daylightSaving ? 3600 : 0)
```

Parameters:

- `utcEpoch` : Unix timestamp in UTC

Return:

- Unix timestamp in local timezone
-

```
void handleGetTime()
```

Location: Line 3160

HTTP: GET /api/time

Purpose: Retrieve current time, time source, and WiFi status

Response:

```
{  
  "epoch": 1718500800,  
  "time": "14:30:45",  
  "timeSource": "ntp",  
  "wifi": true,  
  "rtcPresent": true,  
  "rtcValid": true,  
  "uptime": 345600  
}
```

Fields:

- `epoch` : Current Unix timestamp
- `time` : Human-readable HH:MM:SS format
- `timeSource` : Source ("ntp", "browser", "rtc", or "none")
- `wifi` : WiFi station connected status
- `rtcPresent` : DS3231 hardware detected
- `rtcValid` : RTC has valid backed-up time
- `uptime` : Seconds since boot

Frequency: Web UI polls every 1 second

RTC Functions

DS3231 Hardware RTC

`void initRTC()`

Location: Line 519

Frequency: Once at boot

Purpose: Initialize DS3231 I2C communication and test presence

Operations:

1. Attempt I2C connection at address 0x68
2. If DS3231 detected:
 - Read time from chip
 - Validate epoch (should be > 1 billion)
 - If valid: set `rtcPresent = true, rtcTimeValid = true`
 - Load RTC state from NVS (backup)
 - Call `loadRTCFromDS3231()` to sync internal clock
3. If not detected:

- Set `rtcPresent = false, rtcTimeValid = false`
- Use NVS backup if available

Validation:

- Validates I2C communication
- Checks RTC epoch reasonableness
- Falls back gracefully if hardware absent

`void immediateDS3231Sync()`

Location: Line 540

Frequency: Once per hour + on demand

Purpose: Immediately synchronize internal clock with DS3231 hardware

Operations:

1. If DS3231 not present: skip
2. Read current time from DS3231 via I2C
3. Extract epoch from RTC registers
4. Call `syncInternalRTC(epoch)` to update system time
5. Record sync timestamp
6. Reset 1-hour timer

Used By:

- Main loop (hourly maintenance)
- Manual time sync endpoint
- NTP recovery fallback

`bool loadRTCFromDS3231()`

Location: Line 662

Return: `true` if successful, `false` if RTC absent or error

Purpose: Load backed-up time from DS3231 at boot

Operations:

1. If RTC not present: return `false`
2. Read 7-byte date/time from DS3231 (registers 0x00-0x06)
3. Validate epoch in register range
4. Convert to Unix timestamp
5. Update internal RTC: `internalEpoch = readTime`
6. Set `rtcTimeValid = true`
7. Record load timestamp
8. Return `true`

Fallback Order:

1. DS3231 hardware RTC
2. NVS-backed RTC state
3. Browser-provided time (if > 1 hour old)
4. Epoch 1 billion (safe fallback)

`void syncDS3231FromInternalRTC()`

Location: Line 550

Purpose: Write current internal time back to DS3231 hardware

Operations:

1. If RTC not present: skip
2. Get current epoch from internal RTC
3. Convert to date/time components
4. Write 7 bytes to DS3231 registers (0x00-0x06)
5. Verify write via I2C

6. Record sync timestamp

Used By:

- RTC rebase operation
- NTP sync confirmation

```
void performRTCRebase()
```

Location: Line 562

Frequency: Every 5 minutes

Purpose: Recalibrate internal RTC against hardware/elapsed time

Operations:

1. Get current millis: `now = millis()`
2. Get current internal epoch
3. If 5+ minutes since last rebase:
 - If DS3231 present:
 - Read DS3231 time
 - Compare with internal epoch
 - Calculate drift: `(dsTime - internalEpoch) / 300`
seconds
 - Update `driftCompensation` with exponential averaging
 - Update drift history (rolling 4-sample buffer)
 - Sync DS3231 from internal RTC
 - Reset 5-minute timer

Drift Averaging:

```
driftCompensation = 0.75 * driftCompensation + 0.25 *  
newDrift
```

Accuracy Goal:

- $\pm 50\text{ms}$ per hour (0.005% error)
 - Self-corrects continuously
-

`void saveRTCState()`

Location: Line 621

Purpose: Backup current RTC state to NVS flash

Saved Values:

- `internalEpoch` : Current Unix timestamp
- `lastRTCSync` : Last sync millis() value
- `driftCompensation` : Current drift correction factor
- `timezoneOffset` : User timezone setting

NVS Key: `"rtc_state"`

`void loadRTCState()`

Location: Line 627

Frequency: Once at boot

Purpose: Restore RTC state from NVS backup

Restored Values:

- `internalEpoch`
- `lastRTCSync`
- `driftCompensation`
- `timezoneOffset`

Fallback:

- If NVS key missing: use default (epoch 1 billion)

- If epoch invalid: ignore and use DS3231/browser sync
-

void autoSaveInternalRTC()

Location: Line 641

Frequency: Every 100+ seconds (dirty-flag batched)

Purpose: Periodically backup RTC state to avoid lost time on power loss

Logic:

1. Check if 100+ seconds since last save
2. If dirty flag set (time changed significantly):
 - Call `saveRTCState()`
 - Clear dirty flag
 - Record timestamp
3. Typical pattern: saves every 1-5 minutes with normal drift

Write Optimization:

- NVS writes limited to ~once per minute (vs every loop)
 - Reduces flash wear from ~1000 writes/day to ~60 writes/day
 - Extends flash lifespan from months to years
-

Relay Control Functions

Output Control

inline void setRelayOutput(uint8_t index, bool state)

Location: Line 475

Parameters:

- `index` : Relay index (0-15)
- `state` : `true` = ON, `false` = OFF

Purpose: Set physical relay output with polarity handling

Operations:

1. Check if index valid (< GPIO count)
2. Get GPIO pin number: `pin = gpioConfig.pins[index]`
3. Get polarity: `activeLow = gpioConfig.activeLow[index]`
4. Apply polarity logic:
 - If `activeLow=true`: Write `!state` to pin (LOW = relay on)
 - If `activeLow=false`: Write `state` to pin (HIGH = relay on)
5. Write to GPIO: `digitalWrite(pin, outputValue)`

Example:

- Relay set to ON with `activeLow=true` → GPIO goes LOW
- Relay set to ON with `activeLow=false` → GPIO goes HIGH

`void updateRelayOutputs()`

Location: Line 2358

Frequency: Every 500ms

Purpose: Apply any pending relay state changes

Operations:

1. Calculate desired state for each relay:
 - If manual override active: use `manualState`
 - Else if schedule active: use schedule state
 - Else: use default (OFF)
2. For each relay (0 to `gpioConfig.count`):
 - If desired state ≠ last output:

- Call `setRelayOutput(index, newState)`
- Update `lastRelayOutputs[index]`

3. Clear state-dirty flag

Safety:

- Only updates when state actually changes
- No redundant GPIO writes
- 500ms prevents relay chatter

Manual Control

`void handleManualControl()`

Location: Line 3048

HTTP: `POST /api/relay/{index}/manual`

Purpose: Manually toggle relay on/off (override schedules)

Request Body:

```
{
  "state": true,
  "override": true
}
```

Parameters:

- `state` : Desired relay state (true=ON, false=OFF)
- `override` : Enable manual override (true) or release (false)

Operations:

1. Validate relay index (0-15)
2. Update relay config:
 - `relayConfigs[index].manualOverride = override`

- `relayConfigs[index].manualState = state`

3. Save to NVS
4. Flag schedule cache dirty
5. Return JSON success

Response:

```
{
  "success": true,
  "index": 5,
  "state": true,
  "override": true
}
```

`void handleResetManual()`

Location: Line 3076

HTTP: `POST /api/relay/{index}/manual/reset`

Purpose: Clear manual override and return to schedule control

Operations:

1. Validate relay index
2. Set `relayConfigs[index].manualOverride = false`
3. Save config
4. Return JSON confirmation

Response:

```
{
  "success": true,
  "index": 5,
  "override": false,
}
```

```
"resumedSchedule": true
}
```

Relay Status

`void handleGetRelays()`

Location: Line 3009

HTTP: GET /api/relays

Purpose: Retrieve status of all relays

Response:

```
{
  "count": 16,
  "relays": [
    {
      "index": 0,
      "name": "Relay 1",
      "pin": 15,
      "state": true,
      "manualOverride": false,
      "manualState": false,
      "scheduleActive": true,
      "activeLow": true
    },
    ...
  ]
}
```

Fields per Relay:

- `index` : Relay number (0-15)
- `name` : User-assigned name
- `pin` : GPIO pin number
- `state` : Current physical output state

- `manualOverride` : Manual control active
 - `manualState` : Manual on/off if override active
 - `scheduleActive` : Schedule is triggering ON
 - `activeLow` : Polarity (true=LOW=ON)
-

Schedule Functions

Schedule Definition

`void handleSaveRelay()`

Location: Line 3099

HTTP: POST `/api/relay/{index}/schedule`

Purpose: Update relay name and schedule configuration

Request Body:

```
{
  "name": "Living Room Light",
  "schedules": [
    {
      "enabled": true,
      "startHour": 9,
      "startMinute": 30,
      "startSecond": 0,
      "stopHour": 17,
      "stopMinute": 0,
      "stopSecond": 0,
      "days": 62,
      "monthDays": 0
    },
    ...
  ]
}
```

Parameters:

- `name` : Relay display name (max 15 chars)
- `schedules` : Array of up to 8 schedule objects

Schedule Fields:

- `enabled` : Schedule active/inactive flag
- `startHour` , `startMinute` , `startSecond` : Turn-on time (24-hour format)
- `stopHour` , `stopMinute` , `stopSecond` : Turn-off time
- `days` : Day-of-week bitmask (0x01=Sun, 0x02=Mon, etc.)
- `monthDays` : Monthly day bitmask (bit 0=1st, bit 30=31st)

Operations:

1. Validate JSON structure
2. Validate schedule times (0-23 hours, 0-59 min/sec)
3. Update relay name (truncate if > 15 chars)
4. Replace relay schedule array
5. Save to NVS
6. Update schedule cache
7. Return success

Response:

```
{
  "success": true,
  "index": 0,
  "name": "Living Room Light",
  "schedules": [...],
  "saved": true
}
```

Schedule Processing

```
void processRelaySchedules()
```

Location: Line 2429

Frequency: Every 250ms

Purpose: Evaluate all relay schedules and determine active state

Logic for Each Relay:

1. Get current local epoch and date/time
2. For each of 8 schedules:
 - Check if schedule enabled
 - Check day-of-week match (current day vs bitmask)
 - Check monthly day match (1-31) if specified
 - If day matches:
 - Compare current time vs start/stop times
 - If between start and stop: schedule is ACTIVE
3. OR together all active schedules
4. Update `scheduleActiveCache[relayIndex]`

Time Matching:

```
bool isTimeInRange(uint8_t hour, uint8_t min, uint8_t sec,
                  uint8_t startH, uint8_t startM,
                  uint8_t startS,
                  uint8_t stopH, uint8_t stopM,
                  uint8_t stopS) {
    time_t now = (hour * 3600) + (min * 60) + sec;
    time_t start = (startH * 3600) + (startM * 60) +
startS;
    time_t stop = (stopH * 3600) + (stopM * 60) +
stopS;

    if (start < stop)
```



```
        return (now >= start && now < stop); //
Normal range
    else
        return (now >= start || now < stop); // Wrap
(e.g., 22:00-02:00)
}
```

Schedule Cache:

- Stored in `scheduleActiveCache[MAX_RELAYS]`
- Updated every 250ms
- Prevents redundant schedule evaluation
- Used by `updateRelayOutputs()`

```
void updateScheduleCache()
```

Location: Line 2377

Frequency: Called after schedule changes

Purpose: Rebuild schedule cache after config update

Operations:

1. Clear schedule dirty flag
2. For each relay (0 to `gpioConfig.count`):
 - Call `processRelaySchedules()` immediately
 - Update cache entries
3. Record timestamp
4. Mark relays for output update

Schedule Configuration

```
void handleRelayName()
```

Location: Line 3136

HTTP: POST /api/relay/{index}/name

Purpose: Update relay display name only

Request Body:

```
{
  "name": "Kitchen Light"
}
```

Operations:

1. Validate relay index
2. Truncate name to 15 characters
3. Update relayConfigs[index].name
4. Save to NVS
5. Return JSON

Response:

```
{
  "success": true,
  "index": 2,
  "name": "Kitchen Light"
}
```

Web Server Functions

Server Setup

```
void setupWebServer()
```

Location: Line 2906

Frequency: Once at boot

Purpose: Register all HTTP route handlers

Routes Registered:

Status Pages:

- `GET /` → Root HTML UI
- `GET /relays` → Relay control UI
- `GET /wifi` → WiFi configuration UI
- `GET /time` → Time settings UI
- `GET /gpio` → GPIO pin configuration UI
- `GET /system` → System info UI

API Endpoints (documented below):

- Time, WiFi, NTP, Relay, Schedule, GPIO, System endpoints

Catch-All:

- `GET *` → Returns 404 "Not found"

Server Parameters:

- Port: 80 (HTTP)
- Max concurrent clients: 4 (ESP32 default)
- Keep-alive: 5 seconds

REST API Endpoints

Time API

`GET /api/time`

Response:

```
{
  "epoch": 1718500800,
  "time": "14:30:45",
  "timeSource": "ntp",
  "wifi": true,
  "rtcPresent": true,
  "rtcValid": true,
  "uptime": 345600
}
```

POST /api/time/sync/ntp

Purpose: Manually trigger NTP sync

Response:

```
{
  "success": true,
  "syncing": true,
  "timeSource": "ntp"
}
```

POST /api/time/sync/browser

Body:

```
{
  "epoch": 1718500800
}
```

Purpose: Set time from browser (rejected if > 1 hour difference)

Relay API

GET /api/relays

Response: All relay status (see handleGetRelays)

POST /api/relay/{index}/manual

Body:

```
{
  "state": true,
  "override": true
}
```

Purpose: Manual relay control

POST /api/relay/{index}/manual/reset

Purpose: Clear manual override

POST /api/relay/{index}/schedule

Body: Schedule config (see handleSaveRelay)

POST /api/relay/{index}/name

Body:

```
{
  "name": "New Name"
}
```

```
}
```

WiFi API

GET /api/wifi

Response: WiFi status

POST /api/wifi

Body:

```
{
  "ssid": "MyNetwork",
  "password": "Pass123",
  "stationEnabled": true
}
```

POST /api/wifi/scan/start

Purpose: Start WiFi scan

GET /api/wifi/scan/results

Response: Scan results with RSSI

NTP API

GET /api/ntp

Response: NTP configuration

POST /api/ntp

Body:

```
{
  "server": "time.google.com",
  "port": 123
}
```

POST /api/time/sync/ntp

Purpose: Trigger NTP sync immediately

AP API

GET /api/ap

Response: AP settings (SSID, password)

POST /api/ap

Body:

```
{
  "ssid": "ESP32_Switch",
  "password": "NewPass123"
}
```

GPIO Configuration API

GET /api/gpio

Response:

```
{
  "count": 16,
  "pins": [15, 2, 4, 5, 18, 19, 3, 1, 23, 13, 14, 27,
    26, 25, 33, 32],
  "activeLow": [true, true, true, ...]
}
```

POST /api/gpio

Body:

```
{
  "pins": [15, 2, 4, 5, ...]
}
```

Purpose: Reconfigure all relay pins at once

POST /api/gpio/add

Body:

```
{
  "pin": 12
}
```

Purpose: Add single relay (up to 16)

POST /api/gpio/delete

Body:

```
{
  "index": 5
}
```

Purpose: Remove relay and shift down

POST /api/gpio/polarity

Body:

```
{
  "index": 3,
  "activeLow": false
}
```

Purpose: Toggle relay polarity (active-high vs active-low)

System API

GET /api/system

Response:

```
{
  "uptime": 345600,
  "freeHeap": 65432,
  "heapFragmentation": 12,
  "ntpSyncCount": 14,
  "wifiReconnectCount": 2,
  "lastNTPSync": 1718500000,
  "lastWiFiReconnect": 1718499000,
}
```

```
"firmwareVersion": 9,  
"gpioCount": 16,  
"relayStates": [true, false, true, ...]  
}
```

POST /api/system/reset

Purpose: Reboot ESP32 (all connections lost, relays unchanged)

POST /api/system/factory-reset

Purpose: Factory reset (clears all config, relays default OFF)

POST /api/system/global-active-mode

Body:

```
{  
  "mode": 0  
}
```

Modes:

- **0** : Standard (schedules + manual)
- **1** : Always-on (force all relays ON, ignore schedules)
- **2** : Always-off (force all relays OFF, ignore schedules)

Purpose: Global behavior override

Self-Healing System

Overview

The `SelfHealingSystem` class provides automatic recovery from WiFi, NTP, RTC, and mDNS failures without manual intervention.

Health Tracking:

```
class SelfHealingSystem {
    uint8_t subsystemHealthFlags;    // Bitmask of
    subsystem states
    unsigned long lastHealthCheck;    // Last check
    timestamp
};
```

Subsystem Flags (bit positions):

- Bit 0: WiFi health (1=healthy, 0=failed)
- Bit 1: NTP health
- Bit 2: RTC health
- Bit 3: mDNS health
- Bit 4: Web server health
- Bit 5: DNS server health

Critical State Management

```
void SelfHealingSystem::saveCriticalState()
```

Location: Line 707

Purpose: Backup critical relay states for recovery

Saved Data:

- Active relay count
- Current relay states (all 16)
- Last update timestamp

- CRC32 checksum (corruption detection)

NVS Storage:

- Key: "critical_state"
- Triggers on state change
- Used for graceful recovery after power loss

```
bool SelfHealingSystem::restoreCriticalState()
```

Location: Line 724

Return: `true` if restored, `false` if corrupted/missing

Purpose: Restore relay states from backup

Operations:

1. Load critical state from NVS
2. Verify checksum (CRC32)
3. If checksum valid:
 - Restore relay states
 - Apply saved states to GPIO outputs
 - Return `true`
4. If checksum invalid:
 - Log corruption error
 - Discard backup
 - Return `false`

Safety:

- Prevents state corruption from flash errors
 - Falls back to safe state (all OFF) if failed
-

WiFi Recovery

bool SelfHealingSystem::recoverWiFi()

Location: Line 840

Return: `true` if WiFi connected or recovered

Purpose: Detect and recover from WiFi disconnection

Operations:

1. Check WiFi status
2. If connected: mark healthy, return `true`
3. If disconnected:
 - Increment reconnect attempt counter
 - If attempts < 3:
 - Call `WiFi.reconnect()`
 - Return `false` (recovery in progress)
 - Else:
 - Force full restart: `WiFi.disconnect()` then `WiFi.begin()`
 - Reset attempt counter
 - Return `false`

bool SelfHealingSystem::liveReconfigureWiFi()

Location: Line 747

Return: `true` if reconfiguration successful

Purpose: Update WiFi credentials without full restart

Operations:

1. Retrieve updated SSID/password from config
2. Call `WiFi.begin(ssid, password)`

3. Start 20-second timeout for connection
 4. Return result
-

mDNS Recovery

`void restartAPIfNeeded(bool forceRestart)`

Location: Line 813

Purpose: Ensure AP interface is always active

Operations:

1. Check AP status via `WiFi.softAPgetStationNum()` timeout
 2. If no response for 30 seconds: AP unresponsive
 3. If `forceRestart=true` or AP unresponsive:
 - Disable WiFi
 - Re-enable with `WIFI_AP_STA` mode
 - Restart AP with stored SSID/password
 - Restart mDNS
 4. Record restart timestamp
-

`bool SelfHealingSystem::liveReconfigureMDNS()`

Location: Line 767

Return: `true` if successful

Purpose: Update mDNS hostname without restart

Operations:

1. Stop mDNS service
2. Update hostname in config
3. Restart mDNS with new hostname

4. Return success status

`bool SelfHealingSystem::recoverMDNS()`

Location: Line 853

Return: `true` if mDNS healthy

Purpose: Detect and recover mDNS failures

Operations:

1. Query mDNS for current service
 2. If query fails:
 - Stop mDNS
 - Call `startMDNS()` to restart
 - Return `false`
 3. If healthy: return `true`
-

DNS Server Recovery

`bool SelfHealingSystem::liveReconfigureDNS()`

Location: Line 785

Return: `true` if configuration applied

Purpose: Update DNS settings

Operations:

1. Stop current DNS server
 2. Apply new DNS configuration
 3. Restart DNS server
 4. Return `true`
-

```
bool SelfHealingSystem::recoverDNS()
```

Location: Line 856

Return: `true` if DNS healthy

Purpose: DNS server health check

Operations:

1. Test DNS by processing a request
 2. If server unresponsive: restart it
 3. Return health status
-

Web Server Recovery

```
bool SelfHealingSystem::liveReconfigureWebServer()
```

Location: Line 791

Return: `true` if restart successful

Purpose: Restart web server without losing state

Operations:

1. Store current server state
 2. Stop web server
 3. Re-register all handlers
 4. Restart web server (port 80)
 5. Restore state
 6. Return `true`
-

```
bool SelfHealingSystem::recoverWebServer()
```

Location: Line 859

Return: `true` if web server healthy

Purpose: Check and recover web server

Operations:

1. Test web server by attempting connection
 2. If unresponsive for 10+ seconds:
 - Call `liveReconfigureWebServer()`
 3. Return health status
-

NTP Recovery

`bool SelfHealingSystem::recoverNTP()`

Location: Line 862

Return: `true` if NTP healthy

Purpose: NTP synchronization health check

Operations:

1. Check time since last successful NTP sync
 2. If > 1 hour without sync:
 - Increment failure counter
 - If counter > 3: force server switch
 - Call `tryNTPSync()` immediately
 - Return `false`
 3. If recent sync: return `true`
-

RTC Recovery

`bool SelfHealingSystem::recoverRTC()`

Location: Line 883

Return: `true` if RTC healthy

Purpose: RTC synchronization health check

Operations:

1. If DS3231 hardware present:
 - Read current time from chip
 - Compare vs internal RTC
 - If difference > 5 seconds:
 - Resync internal RTC from DS3231
 - Return `false` (recovered from drift)
 2. Check RTC battery backup validity
 3. Return health status
-

Targeted Recovery

`void SelfHealingSystem::performTargetedRecovery()`

Location: Line 903

Frequency: Called by `smartRecovery()`

Purpose: Recover specific failed subsystems

Operations:

1. Evaluate each subsystem health flag
2. For each failed subsystem:
 - Call targeted recovery function
 - Update health flag
 - Record recovery attempt
3. If recovery successful: clear error counter
4. If recovery fails: increment counter (max 3 retries)

Recovery Targets:

- WiFi (via `recoverWiFi()`)
 - NTP (via `recoverNTP()`)
 - RTC (via `recoverRTC()`)
 - mDNS (via `recoverMDNS()`)
 - DNS (via `recoverDNS()`)
 - Web server (via `recoverWebServer()`)
-

Relay State Verification

`void SelfHealingSystem::verifyRelayStates()`

Location: Line 921

Frequency: Called by `smartRecovery()`

Purpose: Verify relay outputs match expected state

Operations:

1. Load expected state from critical state backup
2. For each relay:
 - Read current GPIO output state
 - Compare vs expected state
 - If mismatch:
 - Reapply correct state
 - Log state correction
3. Update critical state backup
4. Record verification timestamp

Safety:

- Corrects relay states after power glitches
- Detects GPIO pin failures

- Restores scheduled operation
-

Smart Recovery

```
void SelfHealingSystem::smartRecovery()
```

Location: Line 940

Frequency: Every 5-10 seconds (adaptive)

Purpose: Automatic system health monitoring and recovery

Adaptive Logic:

1. Check time since last health check
2. If system healthy:
 - Check interval: 10 seconds
 - Only verify relay states
3. If any subsystem failed:
 - Check interval: 5 seconds (faster recovery)
 - Call `performTargetedRecovery()`
 - Call `verifyRelayStates()`
4. After 3 consecutive recovery failures:
 - Attempt full restart (graceful reboot)
5. Update health metrics

Recovery Sequence:

```
Failure Detected (millis)
↓
performTargetedRecovery() (5 sec retry interval)
↓
If success: clear flags, resume normal
If fail 3x: initiate graceful shutdown & reboot
```

Memory Management

Heap Fragmentation Prevention

```
void performMemoryCleanup()
```

Location: Line 975

Frequency: Every 30 seconds

Purpose: Compact heap and free unused memory

Operations:

1. Retrieve current heap statistics
2. Force garbage collection (Arduino framework)
3. Check for any dangling pointers
4. Deallocate temporary buffers (if any)
5. Update `healthMetrics.memoryFragmentation`
6. Record cleanup timestamp

Fragmentation Calculation:

```
fragmentationPercent = (largestFreeBlock /  
totalFreeHeap) * 100
```

```
void cleanupStaleResources()
```

Location: Line 991

Frequency: Called by main loop

Purpose: Clean up stale/disconnected resources

Operations:

1. Check WiFi connection state

- If lost but `wifiConnecting=true` : clear flag after timeout
2. Check DNS/Web server connections
 - Drop connections idle for 10+ seconds
 3. Clear response cache if older than 30 seconds
 4. Flush NVS if dirty-flag batching limit exceeded
-

`void checkAndCleanMemory()`

Location: Line 1011

Frequency: Every 60 seconds

Purpose: Comprehensive memory health check

Operations:

1. Get current heap statistics
 - `freeHeap` : Available memory
 - `largestFreeBlock` : Largest contiguous block
 - `usedHeap` : Allocated memory
 2. Calculate fragmentation percentage
 3. If fragmentation > 50%:
 - Trigger `performMemoryCleanup()`
 4. If free heap < 10 KB:
 - Log warning
 - Trigger emergency cleanup
 5. If free heap < 5 KB:
 - Log critical error
 - Perform targeted recovery
 6. Update health metrics
-

Web Server Health

```
void checkWebServerHealth()
```

Location: Line 1004

Frequency: Continuous (checked every loop)

Purpose: Monitor web server responsiveness

Operations:

1. Check if client connection exists
 2. If client connected: record activity timestamp
 3. If no activity for 10+ seconds:
 - Flag web server as stale
 - Trigger recovery on next health check
-

Configuration Management

System Configuration

```
void initDefaults()
```

Location: Line 2801

Purpose: Initialize all system settings to factory defaults (covered above)

```
void loadConfiguration()
```

Location: Line 2833

Frequency: Once at boot

Purpose: Load system config from NVS

NVS Keys Loaded:

- "sys_config" : System settings (SSID, NTP, timezone, etc.)
- "relay_0" through "relay_15" : Per-relay configurations
- "gpio_config" : GPIO pin assignment
- "ext_config" : Extended features

Fallback:

- If keys missing: call `initDefaults()`
- If version mismatch: migrate config and save

```
void saveConfiguration()
```

Location: Line 2861

Frequency: Called after any config change

Purpose: Write system config to NVS

Saved Data:

- System config (SSID, NTP server, timezone, DST)
- All 16 relay configs (names, schedules, overrides)
- GPIO configuration (pins, polarity)
- Extended config (global modes)

Write Safety:

- Uses dirty-flag batching
 - Max 1 write per second
 - Reduces flash wear
-

Extended Configuration

```
void loadExtConfig()
```


Location: Line 2867

Frequency: Once at boot

Purpose: Load extended feature config from NVS

Loaded Values:

- `global_active_mode` : Global relay behavior (0-2)
 - Reserved padding: Future extension space
-

`void saveExtConfig()`

Location: Line 2897

Frequency: Called after mode changes

Purpose: Write extended config to NVS

GPIO Configuration

`void loadGPIOConfig()`

Location: Line 2172

Frequency: Once at boot

Purpose: Load GPIO pin configuration from NVS

Loaded Values:

- `pins[0..15]` : GPIO pin numbers
- `count` : Active relay count (1-16)
- `activeLow[0..15]` : Polarity per relay
- `magic` : Validation magic (0xD002)

Validation:

- Check magic number
- Validate pin count (1-16)

- Ensure no duplicate pins

Fallback:

- If invalid: restore defaults (first 16 GPIOs)
-

void saveGPIOConfig()

Location: Line 2187

Frequency: Called after GPIO changes

Purpose: Write GPIO config to NVS

GPIO Dynamic Reconfiguration

void handleGetGPIOConfig()

Location: Line 3513

HTTP: GET /api/gpio

Purpose: Retrieve current GPIO pin assignment

Response:

```
{
  "count": 16,
  "pins": [15, 2, 4, 5, 18, 19, 3, 1, 23, 13, 14, 27,
    26, 25, 33, 32],
  "activeLow": [true, true, true, true, ...]
}
```

void handleSaveGPIOConfig()

Location: Line 3534

HTTP: POST /api/gpio

Purpose: Reconfigure all relay pins and polarity

Request Body:

```
{  
  "pins": [15, 2, 4, 5, ...]  
}
```

Operations:

1. Validate pin array
2. Check for duplicates
3. Preserve relay names and schedules
4. Re-initialize GPIO pins with new assignment
5. Reset relay outputs to safe state (OFF)
6. Save to NVS
7. Update schedule cache

```
void handleAddGPIO()
```

Location: Line 3599

HTTP: POST /api/gpio/add

Purpose: Add a single relay (up to 16 total)

Request Body:

```
{  
  "pin": 12  
}
```

Operations:

1. Check count < 16
2. Check pin not already in use
3. Add new pin to config

4. Create default relay (name = "Relay N")
 5. Set to active-low polarity
 6. Initialize GPIO OUTPUT
 7. Save to NVS
-

`void handleDeleteGPIO()`

Location: Line 3641

HTTP: POST /api/gpio/delete

Purpose: Remove a relay and shift down

Request Body:

```
{
  "index": 5
}
```

Operations:

1. Validate index < count
 2. Set relay to OFF (safety)
 3. Shift remaining relays down (array)
 4. Decrement count
 5. Save to NVS
-

`void handleToggleActiveLow()`

Location: Line 3680

HTTP: POST /api/gpio/polarity

Purpose: Toggle relay polarity (active-low ↔ active-high)

Request Body:

```
{
  "index": 3,
  "activeLow": false
}
```

Operations:

1. Validate index
2. Update polarity: `activeLow[index] = toggle`
3. Reset relay output
4. Save to NVS

Response:

```
{
  "success": true,
  "activeLow": false
}
```

Global Active Mode

`void handleGlobalActiveMode()`

Location: Line 3712

HTTP: `POST /api/system/global-active-mode`

Purpose: Set global relay behavior mode

Request Body:

```
{
  "mode": 0
}
```

Modes:

- 0 : Standard (schedules + manual overrides)
- 1 : Always-on (force all relays ON, ignore everything)
- 2 : Always-off (force all relays OFF, ignore everything)

Operations:

1. Validate mode (0-2)
2. Update `extConfig.global_active_mode`
3. Apply new behavior to all relays
4. Save to NVS
5. Update relay outputs

Response:

```
{
  "success": true,
  "mode": 0
}
```

System Recovery & Health

Factory Reset

```
void checkBootButton()
```

Location: Line 2125

Frequency: Continuous (checked every loop)

Purpose: Monitor boot button (GPIO 0) for factory reset

Operations:

1. Read GPIO 0 (INPUT_PULLUP, active LOW)
2. If button pressed (LOW):
 - Record press start time

- Begin 5-second countdown
 - Visual indication (via serial if needed)
3. If button held for 5+ seconds:
- Trigger factory reset flag
 - Call `handleFactoryReset()`
4. If button released before 5 sec:
- Cancel operation
 - Continue normal operation
-

`void handleFactoryReset()`

Location: Line 3477

HTTP: `POST /api/system/factory-reset`

Purpose: Complete system factory reset

Operations:

1. Clear all NVS data
 - Erase `"sys_config"`
 - Erase `"relay_*" (0-15)`
 - Erase `"gpio_config"`
 - Erase `"ext_config"`
 - Erase `"rtc_state"`
 - Erase `"critical_state"`
2. Call `initDefaults()` to reinitialize
3. Reset all relays to OFF
4. Clear schedule cache
5. Restart WiFi with default SSID/password
6. Optionally restart ESP32 after 2 seconds

Result:

- All configuration erased
 - System reboots to factory state
 - Default AP: "ESP32_16CH_Timer_Switch" / "ESP32-admin"
 - Default relays: 16x OFF, no schedules
-

System Restart

`void handleReset()`

Location: Line 3472

HTTP: POST /api/system/reset

Purpose: Soft reboot (configuration preserved)

Operations:

1. Note that reboot is imminent
2. Save any pending configuration
3. Close all connections gracefully
4. Call `ESP.restart()`

Result:

- WiFi drops, all sockets close
 - Relays maintain last state during ~2-second reboot
 - Configuration preserved
 - Boot sequence repeats
-

System Information

`void handleGetSystem()`

Location: Line 3437

HTTP: GET /api/system

Purpose: Retrieve system health and diagnostics

Response:

```
{
  "uptime": 345600,
  "freeHeap": 65432,
  "heapFragmentation": 12,
  "ntpSyncCount": 14,
  "wifiReconnectCount": 2,
  "lastNTPSync": 1718500000,
  "lastWiFiReconnect": 1718499000,
  "firmwareVersion": 9,
  "gpioCount": 16,
  "relayStates": [true, false, true, false, ...]
}
```

Fields:

- `uptime` : Seconds since boot
- `freeHeap` : Available memory (bytes)
- `heapFragmentation` : Fragmentation percentage (0-100)
- `ntpSyncCount` : Total successful NTP syncs since boot
- `wifiReconnectCount` : WiFi reconnection count
- `lastNTPSync` : Timestamp of last NTP sync
- `lastWiFiReconnect` : Timestamp of last WiFi reconnect
- `firmwareVersion` : Firmware version number
- `gpioCount` : Active relay count (1-16)
- `relayStates` : Current state of all relays

Utility Functions

Time Comparison Helpers

```
inline bool timeHasElapsed(unsigned long current,  
unsigned long previous, unsigned long interval)
```

Location: Line 134

Purpose: Millis-safe elapsed time check (prevents 49-day overflow)

Formula:

```
return (current - previous) >= interval;
```

Usage:

```
if (timeHasElapsed(millis(), lastCheck, 30000)) {  
    // 30+ seconds have passed  
}
```

```
inline bool isTimeReached(unsigned long current, unsigned  
long target)
```

Location: Line 141

Purpose: Millis-safe future time check

Formula:

```
return (long)(current - target) >= 0;
```

GPIO Helpers

```
inline bool isActiveLow(uint8_t index)
```

Location: Line 442

Purpose: Get relay polarity (active-low vs active-high)

Return: `true` if active-low, `false` if active-high

```
inline int getRelayPin(uint8_t index)
```

Location: Line 462

Purpose: Get GPIO pin number for relay

Return: GPIO pin number (or -1 if invalid index)

```
inline uint8_t getActiveRelayCount()
```

Location: Line 468

Purpose: Get number of configured relays

Return: Count (1-16)

NTP Interval Helper

```
inline unsigned long getNTPInterval()
```

Location: Line 457

Purpose: Get NTP retry interval with jitter

Return: Interval in milliseconds (typically 30000 + random offset)

mDNS Hostname

```
String getMDNSHostname()
```

Location: Line 2550

Purpose: Get current mDNS hostname

Return: Hostname string (e.g., "esp32" → resolves as "esp32.local")

```
void setMDNSHostname(const char* hostname)
```

Location: Line 2553

Purpose: Update mDNS hostname

Operations:

1. Validate hostname (alphanumeric, hyphens, < 32 chars)
 2. Update config
 3. Restart mDNS service with new hostname
-

mDNS Service Management

```
void startMDNS()
```

Location: Line 2507

Frequency: Once at boot + on recovery

Purpose: Initialize mDNS service

Operations:

1. Call `ESPmDNS.begin(hostname)`
 2. Add service: `_http._tcp` on port 80
 3. Add service: `_ws._tcp` (WebSocket simulation)
 4. Update TXT records with device info
 5. Set `mdnsStarted = true`
-

```
void stopMDNS()
```

Location: Line 2537

Purpose: Shut down mDNS service

Operations:

1. Call `ESPmDNS.end()`
 2. Set `mdnsStarted = false`
-

`void restartMDNS()`

Location: Line 2543

Purpose: Restart mDNS (stop + start)

Operations:

1. Call `stopMDNS()`
 2. Wait 100ms
 3. Call `startMDNS()`
-

`void scheduleMDNSRestart()`

Location: Line 2546

Purpose: Schedule mDNS restart for next loop iteration

Operations:

1. Set `mdnsRestartScheduled = true`
 2. Record timestamp
 3. On next loop: restart if 2+ seconds passed
-

Configuration Features

NTP Configuration

```
void handleGetNTP()
```

Location: Line 3309

HTTP: GET /api/ntp

Purpose: Retrieve NTP settings

Response:

```
{
  "server": "time.google.com",
  "port": 123,
  "interval": 3600000,
  "lastSync": 1718500000
}
```

```
void handleSaveNTP()
```

Location: Line 3322

HTTP: POST /api/ntp

Purpose: Update NTP server and port

Request Body:

```
{
  "server": "time.windows.com",
  "port": 123
}
```

AP Configuration

```
void handleGetAP()
```

Location: Line 3365

HTTP: GET /api/ap

Purpose: Retrieve AP settings

Response:

```
{
  "ssid": "ESP32_16CH_Timer_Switch",
  "password": "ESP32-admin",
  "ip": "192.168.4.1"
}
```

void handleSaveAP()

Location: Line 3377

HTTP: POST /api/ap

Purpose: Update AP SSID and password

Request Body:

```
{
  "ssid": "NewAPName",
  "password": "NewPassword123"
}
```

Operations:

1. Validate SSID/password lengths
2. Update AP config
3. Restart AP with new credentials
4. Save to NVS
5. Force WiFi restart with new AP params

Key Design Patterns

Millis() Overflow Safety

All timing uses **elapsed-time pattern**:

```
if ((millis() - lastTime) >= interval) { /* action */  
}
```

This handles millis() rollover at 49 days automatically.

Dirty-Flag Batching

Configuration saves use dirty flags to avoid excessive NVS writes:

- Flag set when config changes
- Save triggered on timeout or force
- Reduces flash wear by 95%

Rollover-Safe Scheduling

Time ranges handle wraparound (e.g., 22:00 - 02:00):

```
if (startTime < stopTime)  
    return (now >= start && now < stop); // Normal  
else  
    return (now >= start || now < stop); // Wrap
```

Adaptive Recovery

Self-healing system uses faster check intervals when unhealthy:

- Healthy: 10-second checks
- Unhealthy: 5-second checks
- Failed recovery 3x: full restart

Checksum-Protected State

Critical relay state uses CRC32 checksum to detect corruption from flash errors.

Performance Specifications

Memory Usage

- **Static RAM:** ~8 KB (configs, buffers, globals)
- **Heap:** ~30 KB used, ~80 KB available (typical)
- **Flash:** ~450 KB firmware, ~200 KB NVS allocation

Power Consumption

- **Active (all systems):** ~100-150 mA @ 5V
- **WiFi only:** ~80-120 mA
- **Relay operations:** ~10 mA per relay (5V coil)

Response Times

- **API response:** < 50ms (median)
- **Schedule evaluation:** 5-10ms per 16 relays
- **Relay state change:** < 100ms (physical switch time)
- **NTP sync:** 1-5 seconds (network dependent)

Reliability

- **MTBF:** 7+ years (continuous operation)
- **Flash lifespan:** 19+ years (with write optimization)
- **RTC battery:** 5+ years (DS3231 backup)

Troubleshooting

No WiFi Connection

1. Check AP IP `192.168.4.1` (default AP always active)
2. Verify SSID/password are correct

3. Check WiFi signal (RSSI > -70 dBm ideal)
4. Wait up to 30 seconds for auto-reconnect

Time Not Synchronizing

1. Check WiFi connected (needed for NTP)
2. Check `/api/time` endpoint for current time source
3. If "none": trigger NTP sync via `POST /api/time/sync/ntp`
4. Verify NTP server is reachable

Relays Not Responding

1. Check `/api/relays` for current state
2. If schedule active but relay OFF: check polarity via `/api/gpio`
3. Verify GPIO pin is not in use by other system
4. Check physical relay wiring and power

Frequent WiFi Disconnections

1. Check signal strength (move closer to router)
2. Check for WiFi interference (2.4 GHz congestion)
3. Verify AP password is exactly correct (case-sensitive)
4. Check system uptime for stability pattern

License

GPLv3 - Open Source. See LICENSE file.

Support & Contributions

GitHub: <https://www.github.com/xiv3r>

Document Version: 1.0
Last Updated: June 2026
Firmware Version: 9.0